# Les Guards Angular

## Table des matières

1. Introduction ..... 2
2. Mise en place ..... 2

## 1. Introduction

À l'heure actuelle, rien ne nous empêche, côté Angular, de demander des URLs auquelles on n'est pas censé avoir accès. Par exemple si je veux me connecter sur /dashboard alors que je n'ai pas le droit, je vais probablement me faire jeter par l'API, mais rien ne m'empêche d'effectuer cette requête côté Angular.

C'est là qu'arrivent les Guards, ils vont empêcher l'accès aux URLs côté Front en se plaçant **avant** l'exécution de la Route.

## 2. Mise en place

On va utiliser le CLI pour générer le Guard :

```
ng generate guard guards/auth
# ou en raccourci
ng g g guards/auth
```

Le CLI demande quel type → choisir **CanActivate**.

Il existe plusieurs types de Guards en fonction du rôle que l'on veut leur faire jouer :

| Guard | Interface fonctionnelle | Rôle |
|---|---|---|
| canActivate | CanActivateFn | Protège l'accès à une route |
| canActivateChild | CanActivateChildFn | Protège les routes enfants |
| canDeactivate | CanDeactivateFn | Empêche de quitter une route (formulaire non sauvegardé) |
| canMatch | CanMatchFn | Conditionne le chargement du module/route |

Pour notre cas, canActivate suffit amplement.

Un Guard est une **fonction**, que le CLI a créé :

```
import { CanActivateFn } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  return true;
};
```

Pour l'instant, cette fonction laisse tout le monde passer. Modifions ça :

*On utilise inject() ici au lieu d'un constructeur car un Guard est une FONCTION et pas une classe. Mais le concept d'injection reste exactement le même.*

```
// src/app/guards/auth.guard.ts
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth-service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.getToken() != null) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

Ici on ne renvoie True que si on a un token en session (c'est notre service d'authentification qui nous permet de tester ça). Sinon on redirige vers l'URL de login (à supposer que cette URL existe)

Enfin, dans app.routes.ts, on va placer notre Guard dans la route correspondante, avec son type, comme ceci :

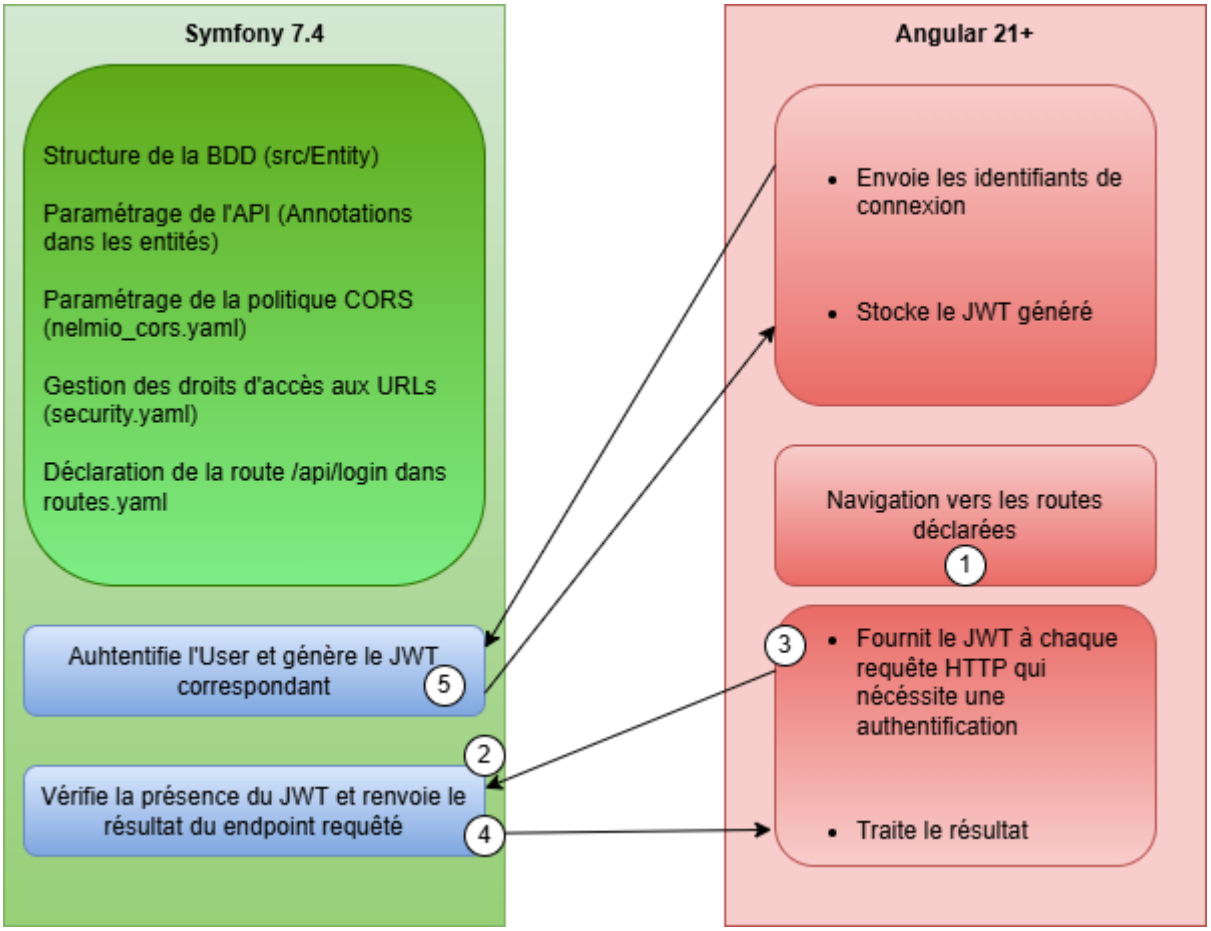
```
export const routes: Routes = [
  { path: '', redirectTo: 'parts', pathMatch: 'full' },
  { path: 'parts', component: PartListComponent },
  {
    path: 'dashboard',
    component: Dashboard,
    canActivate: [authGuard]
  },
];
```

Et voilà, maintenant l'accès à cette route sera également vérifié par Angular, et plus seulement par l'API.

*Une bonne sécurité c'est un assemblage de multitudes de vérifications, autant côté front que côté back.*

# EXERCICE

- Mettre en place des Guards sur les routes part-form, categorie-form, brand-form
- Conditionner le chargement de la liste en page d'accueil par la présence ou non du token
- Reprenez le schéma suivant et indiquez en légende à quoi correspondent chacun de chiffres que vous voyez ☺




1 : C'est le moment où…

2 : C'est ici que Symfony fait…

3 :

4 :

5 :